

Embedded insurance widget: 4-week roadmap

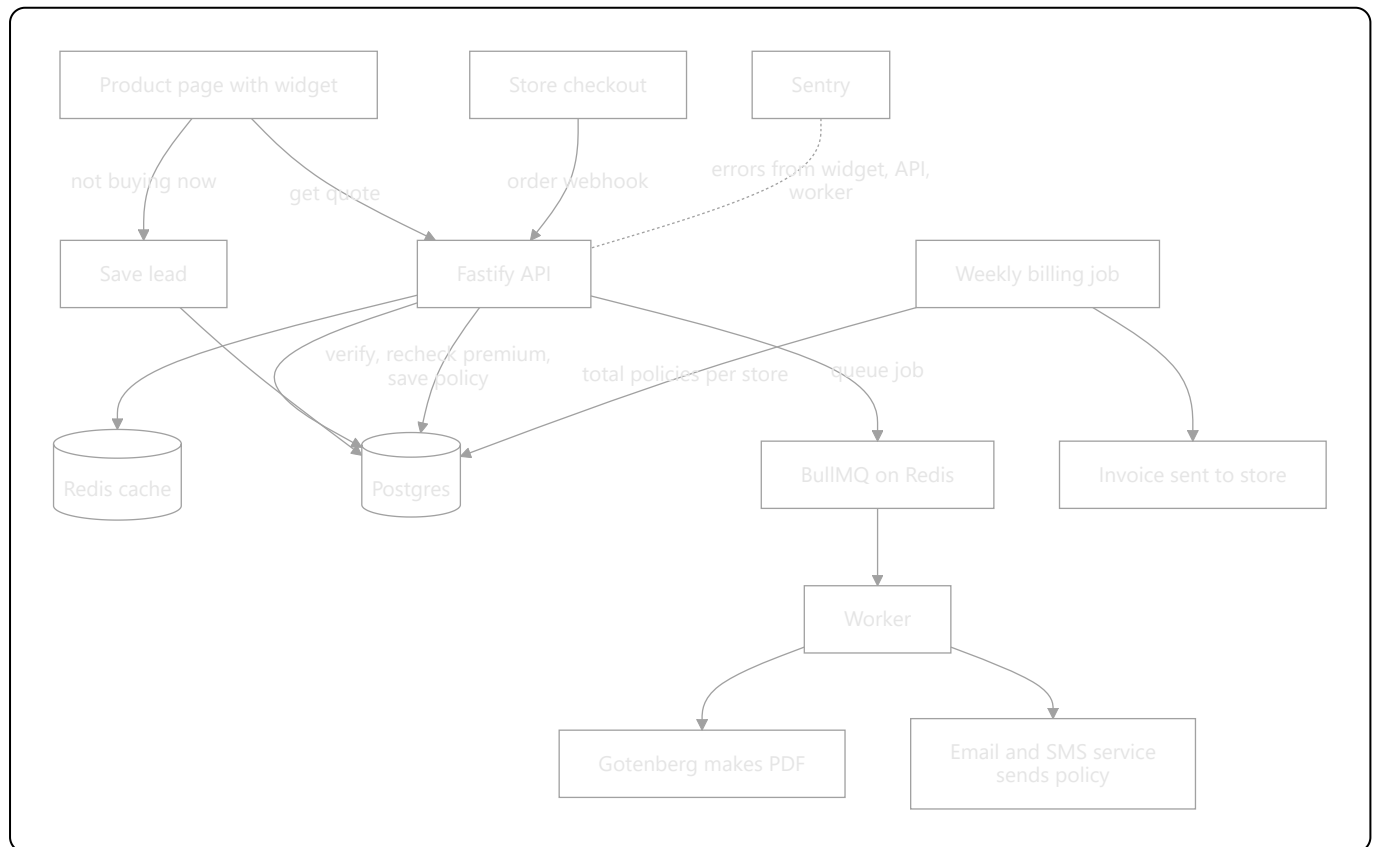
Scope and assumptions

- Shoppers do not create accounts. They see the quote, add insurance to the store's cart and pay at the store's checkout.
- Stores are created manually by our team (admin script) in this phase. A self-serve merchant dashboard is not part of the 4 weeks.
- Insurance is billed to the store weekly, based on the policy records we log.

Stack

Area	Choice
Backend	Fastify with TypeScript
Database	Postgres with Drizzle ORM
Cache	Redis
Job queue and scheduling	BullMQ (on Redis)
PDF generation	Gotenberg (own container)
Email and SMS	In-house
Widget	Plain TypeScript, bundled with esbuild, Shadow DOM
Monitoring	Sentry (errors, performance, cron, uptime)
Deployment	Docker, hosting flexible

System flow



Order of events after a purchase: webhook arrives, request is verified, premium is rechecked, policy is saved, job is queued, store gets its reply, worker makes the PDF and emails it. The weekly job runs separately.

Week 1: foundations and pricing

Goal: the API runs in Docker, the database exists, and `POST /quote` returns a premium.

Day 1: project and Docker

- Create the repo and a Fastify + TypeScript project.
- Write `(docker-compose.yml)` with the API, Postgres, Redis and Gotenberg.
- Add environment config `(.env.example)` and a `(GET /health)` route.
- Done when: `(docker compose up)` starts all four services and `(/health)` returns 200.

Day 2: Sentry and database setup

- Add Sentry to the backend with personal data scrubbing `(beforeSend)`.
- Set up Drizzle and the migration workflow.
- Create the `(stores)` table (name, public key, webhook secret, allowed domains, active flag).
- Done when: a deliberate test error appears in Sentry with no customer data in it, and the migration runs from a clean database.

Day 3: rest of the schema

- Create `quotes`, `policies` (status: pending, sent, failed, voided), `leads` and `invoices` tables.
- Store all money as integers in kobo.
- Add a unique index on store + order id in `policies` to block duplicates.
- Write a seed script that creates one test store.
- Done when: all migrations run and the seed script creates a test store.

Day 4: premium calculation

- Put rating rules in one config file so rates can change without code changes.
- Write the premium function as a pure function and unit test it.
- Build `POST /quote` with JSON Schema validation (product, price, currency).
- Done when: tests pass and a valid request returns a premium while a bad price is rejected.

Day 5: store onboarding and key checks

- Write an admin script that creates a store and generates its `pk_live_...` key and webhook secret.
- Add a check that the public key exists and the request origin is in the store's allowed domains.
- Cache repeated quotes for the same product and price in Redis.
- Done when: a request with a wrong key or wrong domain gets rejected, and the second identical quote comes from cache.

Week 2: embeddable widget

Goal: any page with the snippet shows a live quote and can save a lead.

Day 6: widget skeleton

- Create the widget project and bundle it into a single `widget.js` with esbuild.
- Read `data-key` from the script tag and `data-product` and `data-price` from the `insureongo-offer` div.
- Render a placeholder box inside a Shadow DOM.
- Done when: the snippet on a plain HTML page shows the box and its styling is not affected by the page's CSS.

Day 7: live quote

- Widget calls `POST /quote` and shows the premium.
- Add loading and error states (if the API is down, the widget hides itself and the product page still works).
- Add CORS on the API for allowed store domains.

- Done when: the demo page shows a real premium, and turning off the API does not break the page.

Day 8: add to cart

- Add an "Add insurance" checkbox to the widget.
- When ticked, the widget fires an `insureongo:selected` event carrying the quote id and premium, and calls an optional `onSelect` callback.
- Build a demo store page whose cart listens to this event and adds an insurance line item.
- Done when: ticking the box adds an insurance line to the demo cart, and unticking removes it.

Day 9: leads

- Add a "not buying now" form: email or phone, plus a consent checkbox.
- Build `POST /leads` with validation, saving to Postgres.
- Done when: a submitted lead appears in the `leads` table and the form refuses to submit without consent.

Day 10: widget monitoring and hosting

- Add Sentry to the widget.
- Set up hosting for `widget.js` at `cdn.insureongo.com` with cache headers and a versioned filename.
- Review the week and fix loose ends.
- Done when: the snippet with the CDN URL works on the demo store, and a widget error shows up in Sentry.

Week 3: purchase flow

Goal: a completed order on the demo store produces a saved policy and an email with a PDF.

Day 11: webhook endpoint

- Build `POST /webhooks/order` that verifies the store's HMAC signature using its webhook secret.
- Use the unique store + order id index so a repeated call returns success without creating a second policy.
- Done when: a bad signature gets 401, and sending the same order twice creates one policy.

Day 12: policy record

- Recalculate the premium from the price in the order data, ignoring the browser value.
- Save the policy with status `pending` before doing anything else.

- Done when: a valid webhook creates a `pending` policy with the recalculated premium.

Day 13: queue and worker

- Set up BullMQ and a worker container in `docker-compose.yml`.
- The webhook adds a `send-policy` job and replies to the store immediately.
- Configure retries with increasing delays.
- Done when: the webhook responds fast and the worker picks up the job (a log line is enough for now).

Day 14: PDF and email

- Write the HTML policy template (customer, item, premium, policy number, dates).
- Worker sends the HTML to Gotenberg and gets the PDF back, then stores the file on a Docker volume and saves its path on the policy.
- Send the email with the PDF through the in-house service, then set the policy status to `sent`. On final failure set it to `failed` and report to Sentry.
- Done when: the customer email arrives with a correct PDF, and a forced email failure retries and then shows `failed`.

Day 15: lead follow-ups and full run

- Add a delayed BullMQ job that sends a follow-up to saved leads.
- Run the whole path on the demo store: quote, add to cart, order, webhook, policy, PDF, email.
- Done when: the full path works with no manual steps.

Week 4: billing, hardening, launch

Goal: weekly billing runs on its own and the system is tested and documented.

Day 16: weekly billing job

- Create a BullMQ repeatable job, Monday 06:00 Africa/Lagos time.
- For each store, total the previous week's `sent` policies with a Drizzle query and create an `invoices` row.
- Done when: a manual trigger on seeded data creates correct invoice totals per store.

Day 17: invoice delivery and cron monitor

- Email each store its invoice with a CSV of the policy records.
- Mark invoices as sent and reject double-billing for the same week.
- Add a Sentry cron monitor on the billing job.
- Done when: a store receives the invoice and CSV, and running the job twice for the same week does not bill twice.

Day 18: cancellations and returns

- Add an endpoint or webhook event for cancelled or returned orders.
- Set the policy to `voided` and exclude it from the weekly total (and from an invoice not yet sent).
- Done when: a voided policy does not appear in the invoice.

Day 19: end-to-end testing

- Test the full flow in the Docker setup, including failures: email service down, duplicate webhook, bad signature, wrong domain, Gutenberg down.
- Add Sentry uptime monitoring on the API.
- Fix what breaks.
- Done when: every failure case behaves as intended and shows in Sentry.

Day 20: docs and buffer

- Merchant guide: how to add the snippet and connect the webhook (with the div and script example).
- Developer README: how to run locally, environment variables, how the flow works, where to change rating rules.
- Use any remaining time as buffer.
- Done when: someone who has not seen the project can set up a store from the guide and run the system from the README.

Not in these 4 weeks

- Self-serve merchant dashboard (login, key management, view policies and bills).
- Server health monitoring beyond Sentry.